

Sistema inteligente para monitoreo vehicular con inteligencia artificial

Daniel Felipe Sarmiento Pilonieta
Sebastián Adolfo Albornoz Villamil

Director
Jeison Arley Castillo Bohorquez
MG. En ingeniería

Codirector
Jaime Guillermo Barrero
MG. En ingeniería

Universidad Industrial de Santander
Facultad de Ingenierías Fisicomecánicas
Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones
Bucaramanga
2026

Índice

Objetivo General	6
Objetivos Específicos	6
Introducción	7
Conceptos Previos	9
Inteligencia Artificial y Deep Learning	9
Visión Computacional y Detección de Objetos	9
Modelos de Detección de Objetos	9
Evolución de YOLO	10
Sistemas Embebidos y Edge AI	11
Desarrollo de la Solución	12
Selección del Sistema Embebido	12
Selección del Modelo de Inteligencia Artificial	12
Conjunto de Datos Utilizado	13
Resultados Experimentales	14
Diseño Industrial y Desarrollo de la Carcasa	16
Prueba Experimental de Sellado	19
Gestión Térmica y Diseño Electrónico	20
Aspectos Normativos y Protección de Datos	21
Implementación del Sistema de Monitoreo Vehicular	21
Arquitectura General del Sistema	21
Sistema de Conteo Vehicular	22
Sistema de Clasificación Vehicular	23
Sistema de Estimación de Velocidad	24
Comunicación y Transmisión de Datos	25
Resultados y Análisis	26
Validación del Sistema de Conteo y Clasificación Vehicular	26
Validación del Sistema de Estimación de Velocidad	27
Desempeño Computacional en la Raspberry Pi 5	28
Discusión General	28
Conclusiones	29
Recomendaciones y Trabajo Futuro	31
Referencias	32

Índice de figuras

1.	Comparación de rendimiento entre diferentes modelos YOLO desarrollados por Ultralytics Ultralytics (2025b).	10
2.	Ejecución de los modelos sobre Raspberry Pi 5	15
3.	Modelo tridimensional de la carcasa desarrollado en Onshape	17
4.	Plano técnico de fabricación de la carcasa	18
5.	Comparación entre la primera y la segunda versión de la carcasa	19
6.	Prueba experimental de sellado de la carcasa	20
7.	Esquema eléctrico de conexión de los ventiladores PWM y el sensor DHT11	21
8.	Líneas virtuales utilizadas para el conteo vehicular	23
9.	Esquema de cálculo de velocidad mediante puntos de referencia	24

Índice de tablas

1.	Matriz de criterios para selección del sistema embebido	12
2.	Matriz de criterios para selección del modelo de inteligencia artificial	13
3.	Comparación experimental de los modelos con mejor desempeño	14
4.	Comparación experimental de los modelos con mejor desempeño ejecutados sobre la Raspberry Pi 5	15
5.	Comparación de latencia entre ejecución en computador (Google Colab) y en Raspberry Pi 5	16
6.	Clasificación de protección IP utilizada como referencia	16
7.	Matriz de criterios para selección del material de la carcasa	17
8.	Precisión de conteo inicial (conteo manual vs. sistema)	26
9.	Efecto de los ajustes de calibración sobre la precisión de conteo	27
10.	Validación de la estimación de velocidad con vehículo de referencia	27
11.	Comparación de velocidad de inferencia sobre la Raspberry Pi 5 (formato NCNN)	28

Resumen

Título: Sistema inteligente para monitoreo vehicular con inteligencia artificial

Autores: Daniel Felipe Sarmiento Pilonieta, Sebastián Adolfo Albornoz Villamil

Palabras clave: Movilidad urbana, Inteligencia artificial, Monitoreo vehicular inteligente

Descripción: La gestión eficiente del tráfico urbano representa uno de los mayores retos para las ciudades intermedias en Colombia. En Bucaramanga, la ausencia de sistemas de monitoreo vehicular inteligentes, portátiles y respetuosos de la privacidad ciudadana limita la capacidad de las autoridades para tomar decisiones informadas en tiempo real. Este trabajo presenta el diseño, desarrollo y validación del Sistema Inteligente de Monitoreo Vehicular con Inteligencia Artificial (SIMVIA), un prototipo removible basado en una Raspberry Pi 5 como unidad de procesamiento embebido, una cámara de alta definición y algoritmos del sistema YOLO para la detección, clasificación y conteo de vehículos. La carcasa del dispositivo fue fabricada en acero galvanizado, garantizando su operación en condiciones climáticas adversas. La validación se llevó a cabo en tres fases progresivas: simulación con videos de referencia, pruebas con grabaciones del sitio de despliegue y ensayos con el prototipo físico instalado sobre los puentes peatonales adyacentes al Centro Comercial El Cacique en Bucaramanga. El sistema no almacena imágenes ni videos, cumpliendo con la Ley 1581 de 2012 sobre protección de datos personales. Las pruebas de validación mostraron un reconocimiento de motocicletas cercano al 58 % y errores de estimación de velocidad entre el 0.5 % y el 22 %, dependiendo de las condiciones particulares de cada punto de medición. SIMVIA constituye una contribución tecnológica al avance de las ciudades inteligentes y a la movilidad sostenible en contextos urbanos intermedios.

Abstract

Title: Intelligent vehicle monitoring system with artificial intelligence

Authors: Daniel Felipe Sarmiento Pilonieta, Sebastián Adolfo Albornoz Villamil

Keywords: Urban mobility, Artificial Intelligence, Intelligent vehicular monitoring

Description: Efficient urban traffic management represents one of the greatest challenges for intermediate cities in Colombia. In Bucaramanga, the absence of intelligent, portable, and privacy-respecting vehicle monitoring systems limits the ability of authorities to make informed real-time decisions. This paper presents the design, development, and validation of the Intelligent Vehicular Monitoring System with Artificial Intelligence (SIMVIA), a removable prototype based on a Raspberry Pi 5 as the embedded processing unit, a high-definition camera, and YOLO-based algorithms for vehicle detection, classification, and counting. The device enclosure was manufactured in galvanized steel, ensuring reliable operation under adverse weather conditions. Validation was carried out in three progressive phases: simulation with reference videos, testing with recordings from the deployment site, and field trials with the physical prototype installed on the pedestrian bridges adjacent to the El Cacique shopping mall in Bucaramanga. The system does not store images or videos, complying with Law 1581 of 2012 on personal data protection. SIMVIA represents a technological contribution to the advancement of smart cities and sustainable mobility in intermediate urban contexts.

Objetivo General

Desarrollar un prototipo de sistema inteligente de monitoreo vehicular en tiempo real, mediante técnicas de inteligencia artificial y análisis de datos, que facilite la gestión de la movilidad en entornos urbanos.

Objetivos Específicos

- Seleccionar el hardware adecuado, incluyendo microprocesador, cámara y sistema de alimentación, a partir de una matriz de criterios técnicos que garantice el cumplimiento de los requerimientos del sistema.
- Diseñar e implementar algoritmos de procesamiento de imágenes basados en técnicas de visión computacional (YOLO u otros modelos de detección) para la identificación, conteo y clasificación de vehículos, así como para la estimación del flujo vehicular.
- Desarrollar un prototipo integral del sistema, incorporando aspectos de gestión eficiente de energía, diseño electrónico y consideraciones de diseño industrial que faciliten su instalación y operación en entornos urbanos.
- Realizar pruebas de verificación en campo, instalando el prototipo en una intersección vial de Bucaramanga para evaluar su desempeño en condiciones reales de operación.

Introducción

El crecimiento sostenido del parque automotor ha incrementado la complejidad de la gestión de la movilidad urbana, manifestándose en congestión vial, mayores tiempos de desplazamiento, más emisiones contaminantes y presión sobre la infraestructura de transporte. En Colombia esto es especialmente visible en ciudades intermedias como Bucaramanga, donde el flujo vehicular ha superado la capacidad de los mecanismos tradicionales de monitoreo y control RUNT (2024); el Plan Maestro de Movilidad de Bucaramanga 2022–2037 reconoce, en este sentido, la necesidad de herramientas tecnológicas para analizar información en tiempo real Alcaldía de Bucaramanga (2022).

La disponibilidad de información confiable y oportuna es fundamental para planificar estrategias de movilidad y mejorar la capacidad de respuesta de las autoridades de tránsito. Sin embargo, los sistemas convencionales (sensores inductivos, estaciones fijas de aforo, videovigilancia tradicional) presentan altos costos de instalación y mantenimiento, baja flexibilidad de despliegue y restricciones de tratamiento de datos personales, dado que en Colombia la Ley 1581 de 2012 y el Decreto 1377 de 2013 imponen lineamientos específicos para sistemas que almacenan imágenes o videos identificables Congreso de Colombia (2012); MinCIT (2013).

A nivel internacional, el problema se ha abordado mediante Sistemas Inteligentes de Transporte (ITS) que integran sensores, visión computacional e IA. Ciudades como Singapur, Londres y Barcelona han implementado analítica de video y aprendizaje profundo para mejorar el flujo vehicular y la seguridad vial Gobierno de Singapur (2020); TfL (2021), y en América Latina existen iniciativas similares en Medellín y Bogotá, aunque persisten limitaciones de escalabilidad y adaptación a contextos específicos Ruta N Medellín (2020); Secretaría de Movilidad de Bogotá (2021). Los avances recientes en *edge computing* y en modelos de detección como la familia YOLO (*You Only Look Once*) han ampliado las posibilidades de procesamiento local en tiempo real con buena precisión y eficiencia Katariya, Jannat, Pazho, Noghre, y Tabkhi (2023); Shaqib y cols. (2024).

En este marco, el presente trabajo propone el diseño, desarrollo y validación del *Sistema Inteligente de Monitoreo Vehicular con Inteligencia Artificial* (SIMVIA), orientado a la detección, clasificación y conteo de vehículos en tiempo real mediante procesamiento local de imágenes, buscando una solución robusta, portátil y compatible con condiciones reales de despliegue urbano. El hardware se seleccionó según desempeño, consumo energético, compatibilidad con visión computacional y viabilidad de integración física, resultando en una Raspberry Pi 5 como unidad de cómputo y modelos YOLO como núcleo de procesamiento visual, ejecutando inferencia directamente en el dispositivo sin depender de infraestructura externa.

La propuesta presenta varios elementos diferenciadores: el procesamiento se realiza completamente en el borde, reduciendo la dependencia de la nube; el prototipo es removible y reubicable según las necesidades del entorno urbano; incorpora privacidad por diseño, sin almacenar imágenes ni videos; y fue validado experimentalmente en condiciones reales en una ciudad intermedia colombiana, combinación poco abordada en desarrollos académicos similares UNAL (2023); Univalle (2024); Universidad Javeriana (2023).

Como resultado principal, el proyecto desarrolló un prototipo funcional capaz de detectar, clasificar

y contabilizar vehículos en tiempo real, validado mediante una estrategia de tres fases progresivas: pruebas con videos de referencia, pruebas con grabaciones del sitio de despliegue, y pruebas en campo con el prototipo transportado al entorno de evaluación (sin instalación permanente, dado que el objetivo era la validación experimental).

Los resultados evidencian la viabilidad del sistema para monitoreo vehicular en tiempo real, con desempeño consistente durante las pruebas, y comprueban la efectividad del procesamiento en el borde para generar información útil sin comprometer la privacidad de los usuarios. Este tipo de soluciones puede contribuir al análisis de patrones de movilidad y a la toma de decisiones de las entidades de gestión del tránsito, demostrando que es posible integrar precisión operativa, portabilidad y cumplimiento normativo en un mismo sistema.

El documento se estructura así: la Sección 2 presenta los conceptos previos; la Sección 3 describe la metodología de diseño, implementación y validación; la Sección 4 expone las conclusiones; la Sección 5 formula recomendaciones y trabajo futuro; y la Sección 6 presenta las referencias bibliográficas.

Conceptos Previos

Este capítulo presenta los fundamentos teóricos necesarios para comprender el desarrollo del sistema propuesto: inteligencia artificial, visión computacional, modelos de detección de objetos, entrenamiento de modelos de aprendizaje profundo y sistemas embebidos aplicados a procesamiento en tiempo real.

Inteligencia Artificial y Deep Learning

La Inteligencia Artificial (IA) es el conjunto de técnicas y algoritmos que permiten a los sistemas computacionales realizar tareas que normalmente requieren capacidades humanas, como reconocimiento de patrones, toma de decisiones y aprendizaje automático, con gran relevancia actual en automatización, análisis de imágenes y sistemas inteligentes de transporte IBM (2024); Russell y Norvig (2020). Una de sus principales áreas es el *Machine Learning*, que permite a los sistemas aprender patrones a partir de datos mediante los enfoques supervisado, no supervisado y por refuerzo, sin ser programados explícitamente para cada tarea.

El *Deep Learning* es una subárea del *Machine Learning* basada en redes neuronales profundas capaces de procesar grandes volúmenes de información y extraer características complejas de los datos, con alto desempeño en visión computacional y detección de objetos Goodfellow, Bengio, y Courville (2016); Sarker (2021). Dentro de esta familia, las Redes Neuronales Convolucionales (CNN) son arquitecturas especializadas en el procesamiento de imágenes: utilizan operaciones de convolución para extraer automáticamente bordes, formas y patrones visuales, y son ampliamente utilizadas en sistemas de reconocimiento y detección de objetos en tiempo real LeCun, Bengio, y Hinton (2015).

Visión Computacional y Detección de Objetos

La visión computacional es el campo de la inteligencia artificial enfocado en que las máquinas interpreten información visual proveniente de imágenes o video, automatizando tareas de identificación, análisis y comprensión del entorno visual; en monitoreo vehicular permite detectar automóviles, realizar conteo de tráfico y analizar patrones de movilidad urbana mediante cámaras digitales Szeliski (2022).

Dentro de este campo, la detección de objetos identifica y localiza elementos de interés en una imagen mediante regiones delimitadas (*bounding boxes*), a diferencia de la clasificación de imágenes, que solo determina la categoría principal; esto permite identificar múltiples objetos junto con su ubicación, resultando fundamental para identificar distintos tipos de vehículos en tiempo real Redmon, Divvala, Girshick, y Farhadi (2016). El seguimiento de objetos (*Object Tracking*), por su parte, mantiene la identificación de un objeto a lo largo de múltiples fotogramas de video, permitiendo analizar trayectorias, desplazamientos y comportamiento de vehículos dentro de una escena, y facilitando procesos como el conteo vehicular y el análisis de flujo de tráfico Liu, Lu, Shi, y Ding (2013).

Modelos de Detección de Objetos

YOLO (*You Only Look Once*) es una arquitectura de detección de objetos basada en redes neuronales convolucionales diseñada para tiempo real: a diferencia de modelos con múltiples etapas de procesamiento, YOLO efectúa la detección en una sola pasada sobre la imagen, dividiendo la imagen en una cuadrícula y prediciendo simultáneamente la ubicación, la clase y la probabilidad de cada objeto, lo que

le permite mayores velocidades de inferencia y un equilibrio entre precisión y velocidad ampliamente aprovechado en tráfico inteligente Redmon y cols. (2016).

La Figura 1 presenta una comparación de rendimiento entre diferentes versiones de modelos YOLO desarrollados por Ultralytics.

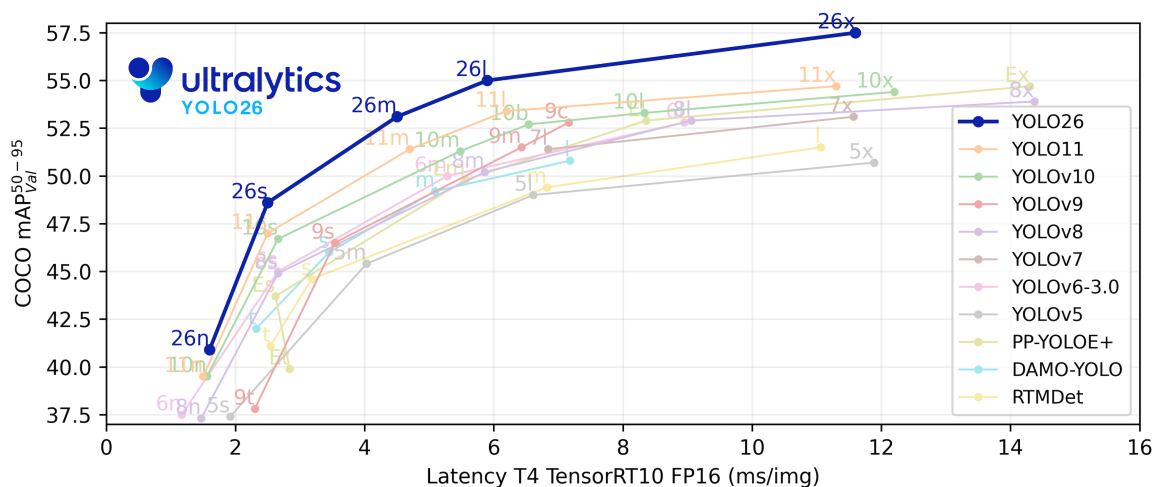


Figura 1

Comparación de rendimiento entre diferentes modelos YOLO desarrollados por Ultralytics (2025b).

Evolución de YOLO

A lo largo de los años, la arquitectura YOLO ha evolucionado significativamente mejorando precisión, velocidad y capacidad de inferencia en dispositivos con recursos limitados. A continuación se describen los cambios concretos introducidos por tres de las versiones más relevantes para este proyecto, y la razón puntual por la cual cada una representó un avance respecto a su predecesora.

YOLOv9 introdujo dos aportes principales: la Información de Gradiente Programable (*Programmable Gradient Information*, PGI) y la red *Generalized Efficient Layer Aggregation Network* (GELAN). El mecanismo PGI mitiga la pérdida de información que ocurre a medida que los datos atraviesan redes neuronales profundas, preservando gradientes relevantes para el aprendizaje sin incrementar el costo computacional en inferencia. GELAN, por su parte, combina las ventajas de arquitecturas previas (CSPNet y ELAN) para lograr una extracción de características más eficiente. Como resultado documentado por sus autores, YOLOv9 alcanza niveles de precisión superiores a los de arquitecturas YOLO anteriores utilizando una cantidad significativamente menor de parámetros entrenables, una mejora relevante para su implementación en dispositivos con recursos computacionales limitados Wang, Yeh, y Liao (2024).

YOLO11, presentado por Ultralytics, incorporó cambios en la estructura de la red troncal (*backbone*) y el cuello (*neck*), reemplazando los bloques C2f utilizados en versiones anteriores por bloques C3k2, más eficientes en el procesamiento de características, y añadiendo bloques C2PSA (*Cross Stage Partial with Spatial Attention*) que permiten a la red enfocar su capacidad de procesamiento en las

regiones más relevantes de la imagen. Según la documentación oficial, estos cambios permitieron reducir el número de parámetros entrenables respecto a versiones anteriores de tamaño equivalente sin sacrificar precisión, además de disminuir la latencia de inferencia. Adicionalmente, YOLO11 unificó dentro de una misma arquitectura el soporte nativo para múltiples tareas de visión artificial (detección, segmentación, estimación de poses, cajas delimitadoras orientadas y clasificación) Ultralytics (2024). **YOLO26**, publicado por Ultralytics durante el desarrollo de este proyecto, fue incorporado al estudio comparativo por tratarse de la arquitectura más reciente disponible al momento de la selección del modelo (véase la Sección de Desarrollo). En las pruebas experimentales realizadas por los autores de este trabajo, comparando directamente contra YOLOv8 bajo las mismas condiciones de entrenamiento y evaluación, YOLO26 obtuvo una velocidad de inferencia aproximadamente 18 % superior, una menor latencia y una reducción en el número de parámetros entrenables, manteniendo niveles de precisión prácticamente equivalentes. Estas mejoras medidas directamente por los autores, y no únicamente reportadas por el fabricante, fueron el criterio determinante para seleccionar YOLO26 como arquitectura definitiva del sistema SIMVIA Ultralytics (2025c).

En síntesis, la relevancia de cada versión no radica en su año de publicación sino en una mejora medible y documentada respecto a su predecesora: PGI/GELAN en YOLOv9 (precisión con menos parámetros), el rediseño de bloques C3k2/C2PSA en YOLO11 (menor latencia y soporte multitarea), y las ganancias de velocidad y eficiencia de YOLO26 verificadas experimentalmente en este mismo proyecto.

El entrenamiento de estos modelos requiere un *dataset*, es decir, un conjunto de datos usado para entrenar y evaluar el modelo, conformado en visión computacional principalmente por imágenes etiquetadas manualmente mediante *bounding boxes* que permiten al modelo aprender los patrones asociados a cada clase; la calidad y diversidad de este conjunto influyen directamente en el desempeño del modelo entrenado Roboflow (2020); Ultralytics (2025a).

Sistemas Embebidos y Edge AI

Un sistema embebido integra hardware y software en una plataforma dedicada para realizar tareas específicas, generalmente con restricciones de memoria, procesamiento y consumo energético; en aplicaciones de inteligencia artificial, permite ejecutar algoritmos de visión computacional directamente sobre dispositivos físicos cercanos a la fuente de datos Innovación Digital 360 (2023); NVIDIA (2025). Este paradigma, conocido como computación en el borde o *Edge Computing*, procesa los datos cerca de su fuente de generación en lugar de depender de servidores remotos, disminuyendo latencia, consumo de ancho de banda y tiempos de respuesta —aspecto especialmente relevante en monitoreo vehicular por la necesidad de inferencia en tiempo real Tan y Cao (2022)—. La implementación de IA en dispositivos embebidos ha permitido así el desarrollo de sistemas autónomos capaces de ejecutar modelos de detección y clasificación sin depender de procesamiento en la nube, combinando técnicas de optimización computacional y arquitecturas eficientes para operar con recursos limitados Bamoumen, Temouden, Benamar, y Chtouki (2022).

Desarrollo de la Solución

Este capítulo describe el proceso de desarrollo del prototipo SIMVIA: las metodologías empleadas, los criterios de selección tecnológica, las decisiones de diseño y las pruebas de validación, de manera que futuros desarrolladores puedan reproducir o mejorar la solución propuesta.

Selección del Sistema Embebido

La selección del sistema embebido se realizó mediante evaluación multicriterio, dado que el dispositivo debía ejecutar algoritmos de inteligencia artificial en tiempo real, transmitir información a plataformas externas y operar de forma autónoma en entornos urbanos. Se evaluaron las alternativas Raspberry Pi 5, Raspberry Pi 4, Jetson Nano, ESP32, Arduino y Ameba, considerando capacidad de procesamiento, memoria RAM, almacenamiento, conectividad, compatibilidad con modelos de IA y costo de implementación.

La Tabla 1 presenta la matriz de criterios utilizada durante el proceso de selección del sistema embebido.

Tabla 1

Matriz de criterios para selección del sistema embebido

Criterio de evaluación	Peso (%)	Raspberry Pi 5	Raspberry Pi 4	Jetson Nano	ESP32	Arduino Uno	Ameba Pro2
Capacidad de procesamiento	30	4.35	3.60	4.40	1.80	1.20	2.40
Memoria RAM	20	4.20	3.40	3.80	1.20	1.00	2.20
Almacenamiento	10	4.30	4.10	3.30	1.20	1.00	2.00
Conectividad	15	4.40	4.10	3.40	4.30	1.60	4.00
Compatibilidad con IA	20	4.40	3.70	4.30	1.30	1.00	2.60
Costo de implementación	5	3.60	4.30	2.40	4.50	4.80	3.40
Puntaje total (ponderado)	100	3.93	3.61	3.70	1.85	1.36	2.68

Nota. Elaboración propia mediante una evaluación multicriterio.

Los resultados mostraron que la Raspberry Pi 5 ofrecía el mejor equilibrio entre potencia de procesamiento, memoria, compatibilidad con librerías de visión artificial, costo, conectividad integrada y capacidad de ejecutar modelos de aprendizaje profundo sin hardware adicional especializado, por lo que fue seleccionada como plataforma principal del prototipo SIMVIA.

Selección del Modelo de Inteligencia Artificial

Para seleccionar la arquitectura de detección de objetos más adecuada, se realizó una evaluación comparativa entre modelos de visión artificial ampliamente utilizados en detección en tiempo real: **YOLOv8**, **YOLOv9**, **YOLO11** y **RT-DETR-L**, en sus versiones *Nano* y *Small* (para YOLO) cuando estaban disponibles.

En la primera etapa, las pruebas en Google Colab indicaron que **YOLOv8** ofrecía el mejor equilibrio entre precisión, velocidad de inferencia y consumo de recursos, por lo que fue seleccionado como arquitectura base inicial. Sin embargo, durante el desarrollo de la investigación **Ultralytics** publicó oficialmente **YOLO26** (14 de enero de 2026), con mejoras en velocidad de inferencia y consumo de recursos manteniendo niveles competitivos de precisión.

Esto hizo necesario ampliar el estudio comparativo incorporando YOLO26, con el fin de determinar si

representaba una alternativa más conveniente para su implementación sobre la Raspberry Pi 5. La comparación se realizó mediante evaluación multicriterio, considerando los aspectos de mayor influencia sobre el desempeño de un sistema embebido de visión artificial en tiempo real:

- Precisión de detección (mAP_{50-95}).
- Velocidad de inferencia (FPS).
- Consumo de memoria RAM.
- Consumo de procesamiento.
- Comportamiento sobre la Raspberry Pi 5.

La Tabla 2 presenta la matriz de criterios utilizada durante el proceso de selección de la arquitectura de inteligencia artificial.

Tabla 2

Matriz de criterios para selección del modelo de inteligencia artificial

Criterio de evaluación	Peso (%)	YOLOv8 Nano	YOLOv8 Small	YOLOv9 Nano	YOLOv9 Small	YOLO11 Nano	YOLO11 Small	YOLO26 Nano	YOLO26 Small	RT-DETR-L
Precisión de detección (mAP)	30	4.40	4.65	4.55	4.70	4.45	4.60	4.55	4.75	4.35
Velocidad de inferencia (FPS)	25	4.30	4.60	4.00	4.30	4.20	4.50	4.70	4.80	3.80
Consumo de memoria RAM (GB)	15	4.40	3.60	4.20	3.40	4.10	3.30	4.60	3.80	2.90
Consumo de procesamiento (CPU/GPU)	15	4.20	3.40	3.90	3.10	3.80	3.00	4.40	3.60	2.60
Comportamiento sobre Raspberry Pi 5	15	4.20	4.10	3.80	3.70	3.90	3.80	4.40	4.20	3.20
Puntaje total (ponderado)	100	4.31	4.40	4.18	4.07	4.15	4.08	4.53	4.55	3.58

Nota. Elaboración propia mediante una evaluación multicriterio.

Los modelos con mejor desempeño en la matriz de evaluación fueron sometidos a una segunda etapa de validación experimental, ejecutada tanto en Google Colab como sobre la Raspberry Pi 5 bajo las mismas condiciones, comparando precisión, velocidad de inferencia, porcentaje de captura vehicular, latencia y parámetros entrenables.

Conjunto de Datos Utilizado

Para el entrenamiento se empleó un conjunto de datos desarrollado en Roboflow Roboflow (2026), compuesto originalmente por **998 imágenes** etiquetadas bajo una única clase, *Vehículo*.

Para mejorar la generalización del modelo frente a distintas condiciones de iluminación, orientación y ruido, se aplicaron herramientas de aumento de datos (*Data Augmentation*) de Roboflow:

- Orientación automática de las imágenes.
- Redimensionamiento a 640×640 píxeles.
- Rotaciones de 90° , 180° y 270° .
- Volteo horizontal y vertical.
- Aplicación de desenfoque (*Blur*) de hasta 1.3 píxeles.

Tras este proceso, el conjunto de entrenamiento quedó conformado por **2238 imágenes**, distribuidas automáticamente por Roboflow así:

- Entrenamiento: 2083 imágenes (87 %).
- Validación: 200 imágenes (8 %).
- Prueba: 100 imágenes (4 %).

El tamaño de entrada de 640×640 píxeles corresponde a la resolución de entrenamiento de los modelos YOLO26 publicados por Ultralytics, lo que permite aprovechar sus pesos preentrenados.

Durante el desarrollo se realizaron experimentos adicionales variando el tamaño del conjunto de entrenamiento, el número de clases, la altura de la cámara y el ángulo de captura. Estos mostraron que incrementar en exceso el número de imágenes o de clases no mejoraba necesariamente la precisión, y sí aumentaba el tiempo de entrenamiento y el consumo de recursos; por el contrario, conjuntos demasiado pequeños reducían la capacidad de generalización y la tasa de detección.

Resultados Experimentales

Los modelos con mejor desempeño se entrenaron bajo las mismas condiciones y se evaluaron mediante precisión (mAP_{50-95}), porcentaje de captura vehicular, F1-Score, velocidad de inferencia (FPS), latencia y número de parámetros entrenables.

Los resultados obtenidos se presentan en la Tabla 3.

Tabla 3

Comparación experimental de los modelos con mejor desempeño

Modelo	mAP_{50-95}	% Captura	F1-Score	FPS	Latencia (ms)	Parámetros (M)	Score
YOLO26s	0.6309	90.48	0.9236	74.14	13.49	9.95	0.9143
YOLOv8s	0.6235	92.15	0.9305	62.85	15.91	11.14	0.8928
YOLO11s	0.6174	91.38	0.9335	74.19	13.48	9.43	0.8749

Las diferencias de precisión entre YOLO26s y YOLOv8s fueron pequeñas, con resultados muy similares en exactitud. Sin embargo, YOLO26s presentó una velocidad de inferencia aproximadamente 18 % superior, menor latencia y menos parámetros entrenables que YOLOv8s, ventajas significativas para su implementación en dispositivos embebidos con recursos limitados como la Raspberry Pi 5.

Con base en estos resultados, YOLO26 fue seleccionada como familia de modelos definitiva por su mejor equilibrio entre precisión, velocidad y eficiencia computacional. La comparación se realizó con las versiones *Small*, por ofrecer más métricas comparables entre arquitecturas; para el despliegue final se optó por la versión *Nano* (YOLO26n), exportada a NCNN por su menor tamaño y consumo de recursos, más adecuada para la ejecución continua en un dispositivo embebido. Su desempeño final se presenta en el Capítulo de Resultados y Análisis.

La evidencia de entrenamiento, validación, curvas de aprendizaje, matrices de confusión y archivos generados está disponible en el repositorio oficial Sarmiento Pilonieta y Albornoz Villamil (2026b) y en el repositorio de anexos Sarmiento Pilonieta y Albornoz Villamil (2026a).

La Figura 2 presenta la ejecución del modelo seleccionado sobre la Raspberry Pi 5, validando su correcto funcionamiento bajo las condiciones reales de operación del prototipo.



Figura 2
Ejecución de los modelos sobre Raspberry Pi 5

Nota. Fotografía de autoría propia correspondiente a la ejecución de los modelos sobre la Raspberry Pi 5 durante las pruebas experimentales realizadas para el desarrollo del sistema SIMVIA.

Para verificar si el comportamiento observado en Colab se mantenía sobre el dispositivo embebido, los tres modelos *Small* de la Tabla 3 (exportados a NCNN, 320×320 píxeles) se sometieron al mismo *benchmark* usado para la selección final (100 fotogramas en vivo). La Tabla 4 presenta los resultados.

Tabla 4
Comparación experimental de los modelos con mejor desempeño ejecutados sobre la Raspberry Pi 5

Modelo	FPS	Latencia (ms)	RAM adicional (MB)
YOLO26s	24.7	40.5	80.9
YOLOv8s	23.4	42.6	66.5
YOLO11s	24.3	41.2	41.4

Nota. Elaboración propia. Prueba ejecutada sobre la Raspberry Pi 5 con el servicio de producción detenido, usando los mismos modelos *Small* evaluados en la Tabla 3, exportados a formato NCNN con tamaño de entrada 320×320 píxeles.

La Tabla 5 contrasta la latencia obtenida en Colab frente a la medida en la Raspberry Pi 5 para los tres modelos.

Tabla 5

Comparación de latencia entre ejecución en computador (Google Colab) y en Raspberry Pi 5

Modelo	Latencia PC (ms)	Latencia Pi (ms)	Incremento
YOLO26s	13.49	40.5	+200 %
YOLOv8s	15.91	42.6	+168 %
YOLO11s	13.48	41.2	+206 %

Nota. Elaboración propia, a partir de los datos de las Tablas 3 y 4.

Como era de esperarse, la latencia aumentó considerablemente (168 %-206 %) al pasar de un entorno con GPU (Colab) a un dispositivo embebido sin unidad gráfica dedicada. Sin embargo, el orden relativo de desempeño se mantuvo igual en ambos entornos —YOLO26s el más rápido, YOLOv8s el más lento—, lo que refuerza la validez de la comparación en Colab como criterio de selección de arquitectura.

Diseño Industrial y Desarrollo de la Carcasa

Dado que SIMVIA fue concebido para operar en exteriores, fue necesario diseñar una carcasa que protegiera los componentes electrónicos de lluvia, humedad, radiación solar, polvo y variaciones de temperatura, garantizando además su mantenimiento y portabilidad.

Como primera etapa se revisaron los grados de protección IP (*Ingress Protection*) utilizados en dispositivos exteriores, estableciendo como objetivo de diseño el grado **IP55**: protección contra polvo en cantidades que no afectan el funcionamiento y contra chorros de agua desde cualquier dirección. La Tabla 6 presenta la clasificación utilizada como referencia.

Tabla 6

Clasificación de protección IP utilizada como referencia

1 ^{er} numeral	Descripción	2 ^{do} numeral (protección contra agua)										
		0	1	2	3	4	5	6	7	8	9	9k
0	No protegido	IP00	IP01	IP02								
1	Objetos sólidos >50 mm	IP10	IP11	IP12	IP13							
2	Objetos sólidos >12 mm	IP20	IP21	IP22	IP23							
3	Objetos sólidos >2,5 mm	IP30	IP31	IP32	IP33	IP34						
4	Objetos sólidos >1 mm	IP40	IP41	IP42	IP43	IP44	IP45	IP46				
5	Protegido contra polvo				IP53	IP54	IP55	IP56				
6	Totalmente protegido contra polvo						IP65	IP66	IP67	IP68	IP69	IP69k

Nota. 1^{er} numeral (protección contra personas y objetos sólidos): 0 = no protegido; 1 = objetos mayores a 50 mm; 2 = objetos mayores a 12 mm; 3 = objetos mayores a 2,5 mm; 4 = objetos mayores a 1 mm; 5 = protegido contra polvo (entrada máxima de aire equivalente a 80 veces el volumen de la cabina); 6 = totalmente protegido contra polvo. 2^{do} numeral (protección contra ingreso perjudicial de agua): 0 = no protegido; 1 = goteo vertical de agua; 2 = goteo hasta 15° de la vertical; 3 = rociado hasta $\pm 60^\circ$ de la vertical; 4 = rociado en todas las direcciones; 5 = chorros de agua en todas las direcciones; 6 = olas de mar o golpes de agua potentes; 7 = protegido contra inmersión; 8 = protegido contra submersión; 9 = protegido contra inmersión (1 min) y resistencia a presión; 9k = protegido contra vapores de agua y alta presión. Adaptado de *Conoce los tipos y grados de protección IP*, por TRACTIAN. Disponible en <https://tractian.com/es/blog/conoce-los-tipos-y-grados-de-proteccion-ip>.

Posteriormente se realizó una evaluación multicriterio para seleccionar el material de fabricación, considerando acero galvanizado, acero inoxidable AISI 304, aluminio 6061 y materiales de impresión 3D (PETG, ASA, ABS), evaluados en resistencia mecánica, resistencia a la corrosión, comportamiento

frente a la intemperie, facilidad de fabricación, peso, costo y mantenimiento.

Los pesos de cada criterio se definieron priorizando los aspectos que más influyen en la vida útil del prototipo en exteriores. La Tabla 7 presenta la matriz utilizada.

Tabla 7

Matriz de criterios para selección del material de la carcasa

Criterio de evaluación	Peso (%)	Acero galvanizado	Acero inoxidable 304	Aluminio 6061	PETG (impresión 3D)	ASA (impresión 3D)	ABS (impresión 3D)
Resistencia mecánica	30	4.80	4.60	4.20	2.00	2.50	2.30
Resistencia a la corrosión	20	4.40	4.80	3.80	2.00	2.50	2.20
Resistencia a la intemperie	20	4.60	4.80	4.20	2.20	3.20	2.50
Facilidad de fabricación	10	3.80	3.20	4.20	4.50	4.10	4.30
Peso	10	3.00	2.50	4.80	4.20	3.80	3.50
Costo	5	4.20	2.40	3.20	4.80	4.20	4.50
Facilidad de mantenimiento	5	4.40	4.40	3.60	2.50	3.20	2.80
Puntaje total (ponderado)	100	4.42	4.07	4.20	2.77	3.24	2.98

Nota. Elaboración propia mediante una evaluación multicriterio.

El acero galvanizado ofreció el mejor equilibrio entre resistencia mecánica, comportamiento frente a la intemperie, mantenimiento y costo. El acero inoxidable tuvo un desempeño ligeramente superior en resistencia a la corrosión, pero su mayor costo reducía su viabilidad; los materiales de impresión 3D (PETG, ASA, ABS) presentaron limitaciones de resistencia estructural y degradación por radiación ultravioleta en exteriores.

Se seleccionó por ello el acero galvanizado como material principal, tomando como referencia su uso en semáforos, gabinetes eléctricos y otros elementos urbanos expuestos permanentemente a la intemperie; su diseño tridimensional se desarrolló en la plataforma CAD *Onshape*, definiendo dimensiones generales, distribución interna de componentes, puntos de fijación, sistema de ventilación y mecanismos de acceso para mantenimiento.

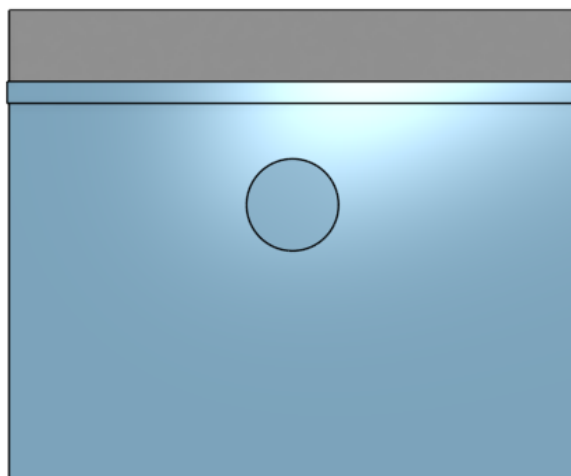


Figura 3

Modelo tridimensional de la carcasa desarrollado en Onshape

Nota. Modelo CAD de autoría propia desarrollado en la plataforma Onshape como parte del proceso de diseño mecánico del prototipo SIMVIA.



Figura 5

Comparación entre la primera y la segunda versión de la carcasa

Nota. Fotografía de autoría propia correspondientes a las dos versiones fabricadas durante el desarrollo del prototipo SIMVIA. La segunda versión incorpora mejoras relacionadas con el sistema de sellado, el mecanismo de cierre, el diseño de las bisagras y el acabado superficial.

Prueba Experimental de Sellado

Para verificar el sellado de la carcasa se realizó una prueba de resistencia al ingreso de agua, exponiendo la carcasa ensamblada a un flujo continuo de agua durante tres minutos desde distintas direcciones, simulando lluvia intensa y salpicaduras propias de la operación en exteriores. Se ubicó un pañuelo de papel seco en el interior antes de la prueba; al finalizar, permaneció completamente seco, sin evidencia de ingreso de agua al compartimiento electrónico.

El resultado confirmó la efectividad del sellado, en línea con el objetivo de diseño IP55, aunque esta prueba es experimental y no un ensayo de certificación conforme a la norma IEC 60529.

La evidencia fotográfica y audiovisual de la prueba de sellado se encuentra disponible en los anexos digitales del proyecto Sarmiento Pilonieta y Albornoz Villamil (2026a).



Figura 6

Prueba experimental de sellado de la carcasa

Nota. Fotografía de autoría propia correspondiente a la prueba experimental de resistencia al ingreso de agua realizada sobre la primera versión de la carcasa. La verificación del sellado se efectuó mediante la inspección de un pañuelo de papel ubicado en el interior del gabinete, el cual permaneció completamente seco al finalizar el ensayo.

Gestión Térmica y Diseño Electrónico

La ejecución continua de inferencia sobre la Raspberry Pi 5, dentro de una carcasa cerrada expuesta a radiación solar, implica riesgo de acumulación de calor que afecta tanto el rendimiento del procesador (*thermal throttling*: reducción automática de frecuencia ante temperaturas elevadas) como la vida útil de los componentes. Por ello se diseñó un sistema de gestión térmica que garantizara una temperatura de operación adecuada sin comprometer la hermeticidad de la carcasa: dos ventiladores controlados por modulación de ancho de pulso (PWM) en los pines GPIO12 y GPIO13, que ajustan continuamente su velocidad en vez de depender de una señal binaria de encendido/apagado, junto con un sensor DHT11 (GPIO4) que monitorea temperatura y humedad para alimentar el control automático.

El sistema implementa una curva de control escalonada que ajusta el porcentaje de velocidad de los ventiladores en función de la temperatura medida:

- Temperatura menor a 40 °C: ventiladores apagados (0 %).
- Entre 40 °C y 50 °C: velocidad del 30 %.
- Entre 50 °C y 65 °C: velocidad del 60 %.
- Entre 65 °C y 75 °C: velocidad del 85 %.
- Temperatura mayor o igual a 75 °C: velocidad máxima (100 %).

Este control gradual mantiene la temperatura en rangos seguros durante la mayor carga computacional, evitando encendidos/apagados abruptos que desgastarían los ventiladores e incrementarían el

consumo energético. El sistema permite además un modo manual vía la API REST, útil durante mantenimiento o pruebas de campo.

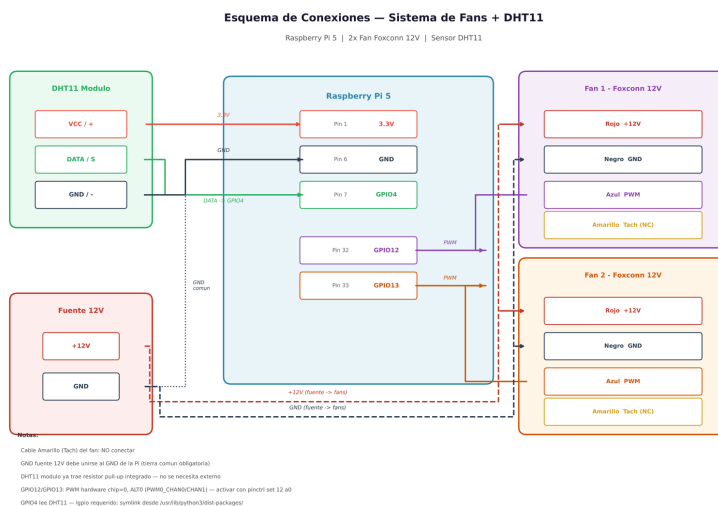


Figura 7

Esquema eléctrico de conexión de los ventiladores PWM y el sensor DHT11

Aspectos Normativos y Protección de Datos

Uno de los objetivos fundamentales del proyecto fue garantizar el cumplimiento de los principios de protección de datos personales. A diferencia de los sistemas tradicionales de videovigilancia, SIMVIA procesa la información localmente: las imágenes capturadas se usan únicamente para ejecutar los algoritmos de detección y clasificación vehicular, y una vez procesadas se transforman en datos estadísticos y numéricos, sin almacenamiento permanente de imágenes o videos.

Este enfoque reduce los riesgos asociados al tratamiento de información personal y responde directamente a la Ley 1581 de 2012 y el Decreto 1377 de 2013 sobre protección de datos personales en Colombia Congreso de Colombia (2012); MinCIT (2013), que restringen el tratamiento de imágenes o videos que permitan identificar personas —una limitación de los sistemas convencionales de videovigilancia identificada desde la concepción del proyecto (véase la Introducción).

Implementación del Sistema de Monitoreo Vehicular

Arquitectura General del Sistema

La arquitectura de software se estructura alrededor de un backend en Python con FastAPI, ejecutado permanentemente en la Raspberry Pi 5 mediante un servicio systemd con reinicio automático ante fallos, y organizado en dos hilos independientes: uno de captura, que lee continuamente los fotogramas de la cámara, y uno de inferencia, que ejecuta el modelo YOLO y el algoritmo de seguimiento sobre cada fotograma sin bloquear la captura de video.

Cada fotograma pasa por detección, clasificación, seguimiento y, cuando corresponde, cálculo de conteo y velocidad. Los resultados agregados (conteo acumulado, flujo vehicular, clasificación por tipo, velocidad promedio) se exponen en un dashboard web vía WebSocket, sin recargar la página.

El sistema no almacena imágenes ni video: cada fotograma se descarta tras analizarse, conservando solo los datos numéricos derivados. Localmente se mantienen un CSV histórico (una fila cada 30 s) y un archivo de estado que se sobrescribe con la misma periodicidad, usado para recuperar el conteo acumulado ante un reinicio.

Cada 30 segundos el sistema también transmite un resumen a una hoja de cálculo en la nube, permitiendo su consulta remota sin acceder directamente al dispositivo (véase la sección de Comunicación y Transmisión de Datos, más adelante en este capítulo).

Sistema de Conteo Vehicular

El sistema realiza el conteo utilizando dos puntos de referencia establecidos mediante mediciones físicas directas en el sitio durante la instalación; el conteo se genera cuando un vehículo atraviesa completamente la región definida entre ambos.

Para evitar contar dos veces al mismo vehículo, el sistema usa el algoritmo de seguimiento ByteTrack, que asigna a cada objeto un identificador (ID) mientras logra asociar sus detecciones entre fotogramas consecutivos.

El sistema registra el instante en que el centro del cuadro delimitador de un vehículo cruza la Línea A y luego la Línea B; solo se contabiliza si su ID cruza ambas líneas en ese orden, lo que permite reconocer el sentido de interés e ignorar vehículos en sentido contrario. Un registro interno de identificadores ya contados evita que un mismo vehículo se cuente más de una vez mientras permanece detectado dentro de la zona de conteo.

Durante las pruebas de campo se identificó que la superposición visual entre vehículos puede provocar reasignación del ID tras una oclusión, comportamiento inherente al algoritmo de seguimiento. También se observó que vehículos estacionados cerca de la franja de conteo (zonas de cargue o parqueo temporal) podían generar conteos ficticios por pequeñas variaciones del cuadro delimitador entre fotogramas. Para mitigar esto se incorporó un parámetro de calibración que excluye del conteo cualquier detección fuera del carril de circulación, sin importar su posición respecto a las líneas A y B.

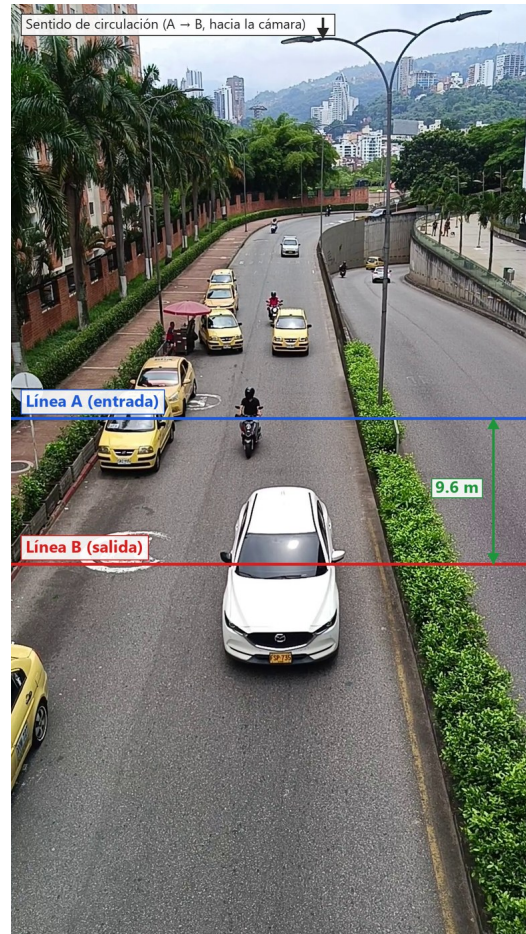


Figura 8

Líneas virtuales utilizadas para el conteo vehicular

Sistema de Clasificación Vehicular

En una primera fase, el sistema clasificaba los vehículos en dos categorías: automóvil (autos, camionetas y furgones) y motocicleta. Para obtener una caracterización más útil para la gestión de movilidad urbana, el conjunto se amplió a cuatro categorías:

- Automóvil.
- Motocicleta.
- Bus.
- Camión.

Esta ampliación respondió a que dichas categorías presentan comportamientos de tránsito diferenciados, relevantes para indicadores como capacidad vial, composición vehicular y nivel de servicio. Se evaluó incorporar categorías adicionales (bicicletas, vehículos de tracción animal), pero un número

mayor de clases incrementaba la complejidad del entrenamiento y reducía el desempeño general sin beneficio claro, por lo que se mantuvieron las cuatro categorías.

Durante la validación se identificó que la clase motocicleta tenía un recall considerablemente inferior (24 % inicial), debido a su menor tamaño relativo y mayor variabilidad de poses. Para mitigarlo se estableció un umbral de confianza diferenciado (0.25 frente a 0.5 para las demás clases) y se reentrenó el modelo con imágenes adicionales de motocicletas corregidas manualmente, lo que permitió incrementar su recall hasta aproximadamente 58 %.

Sistema de Estimación de Velocidad

La estimación de velocidad usa los mismos dos puntos de referencia: la distancia física entre ellos, medida en sitio con un instrumento convencional (10 metros en el punto de instalación definitivo, sobre el puente peatonal del Centro Comercial Cacique), y el tiempo que tarda cada vehículo en recorrerla, obtenido dividiendo el número de fotogramas entre el cruce de ambas líneas por la tasa de fotogramas por segundo. La velocidad se calcula como distancia/tiempo, expresada en km/h.

Para validar esta estimación se registró un vehículo de referencia a velocidad conocida (verificada por velocímetro y GPS) bajo las mismas condiciones de instalación de la cámara, en distintos puntos representativos de la ubicación final. Los errores de estimación oscilaron entre 0.5 % y 22 % respecto a la velocidad real, según las condiciones de cada prueba.

Los errores más altos no correspondían a limitaciones del modelo, sino al algoritmo de seguimiento y a la calibración de cada punto: en tráfico congestionado, la pérdida temporal de un vehículo por oclusión puede reasignar su ID y generar tiempos de cruce artificialmente cortos (velocidades sobrestimadas); vehículos estacionados sin el filtro de zona de parqueo calibrado pueden generar cruces ficticios con velocidades subestimadas. Esto evidenció la importancia de calibrar de forma independiente cada punto, tanto la distancia entre líneas como la exclusión de zonas ajenas al carril.

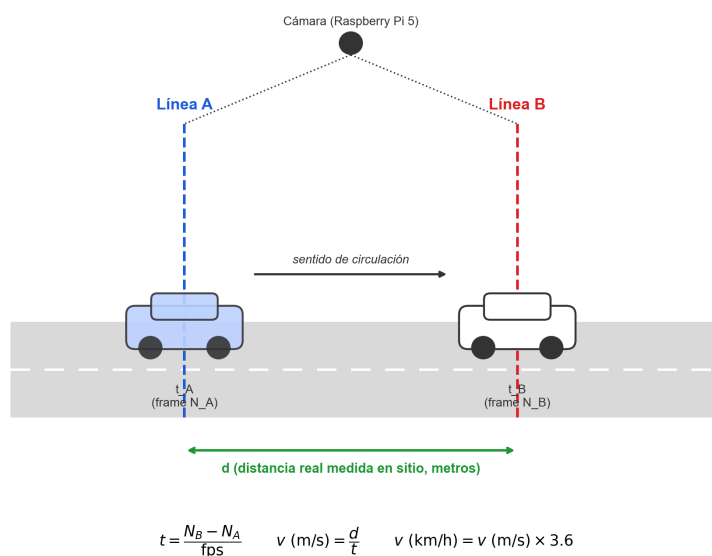


Figura 9

Esquema de cálculo de velocidad mediante puntos de referencia

Comunicación y Transmisión de Datos

La comunicación se estructura en dos niveles. La Raspberry Pi 5 expone una API REST (FastAPI) que permite consultar y modificar la configuración (calibración, líneas de conteo, control de ventiladores), reiniciar contadores y descargar el historial en CSV. Además expone dos canales WebSocket: uno para transmitir en tiempo real el video anotado y las estadísticas del panel, y otro exclusivo para la calibración interactiva de las líneas virtuales, durante la cual la inferencia se pausa mientras el operador ajusta los parámetros.

Adicionalmente, cada 30 segundos el sistema transmite un resumen (marca de tiempo, flujo vehicular estimado, vehículos detectados, conteo por tipo y velocidad promedio) a una hoja de cálculo en la nube, mediante una solicitud HTTP a un script publicado como aplicación web (Google Apps Script). La información se organiza en dos pestañas: una con el estado más reciente y otra con el historial acumulado, para su consulta remota sin acceder directamente al dispositivo.

Resultados y Análisis

Este capítulo presenta los resultados de las dos primeras fases de validación experimental del sistema SIMVIA: pruebas con videos de referencia en condiciones controladas y pruebas con grabaciones del sitio de despliegue (puentes peatonales adyacentes al Centro Comercial El Cacique), analizando la precisión de conteo y clasificación vehicular, la exactitud de estimación de velocidad y el desempeño computacional sobre la Raspberry Pi 5.

Validación del Sistema de Conteo y Clasificación Vehicular

Para validar el conteo se realizó un conteo manual de referencia sobre seis videos del sitio de despliegue, considerando solo los vehículos en el sentido de interés (cruce de Línea A a Línea B). La Tabla 8 presenta los resultados con YOLO26 antes de los ajustes descritos en Desarrollo.

Tabla 8

Precisión de conteo inicial (conteo manual vs. sistema)

Video	Conteo manual	Conteo del sistema	Precisión
112451	12	11	91.7 %
112535	23	15	65.2 %
112653	31	34	90.3 %
112753	25	18	72.0 %
113819	22	17	77.3 %
113930	22	8	36.4 %
Total	135	103	76.3 %

El análisis por clase reveló que, si bien la detección de automóviles alcanzaba cerca del 100 %, la clase motocicleta presentaba un recall notablemente inferior, cercano al 24 %, debido a su menor tamaño relativo en la imagen y a la mayor variabilidad de sus poses respecto a otras categorías.

A partir de esto se aplicaron dos ajustes descritos en el Capítulo de Desarrollo: un umbral de confianza diferenciado para motocicletas y la exclusión de detecciones fuera del carril de circulación mediante el parámetro de calibración correspondiente. La Tabla 9 resume su efecto sobre los videos en que fueron aplicados y validados.

Tabla 9*Efecto de los ajustes de calibración sobre la precisión de conteo*

Video	Precisión inicial	Precisión ajustada	Ajuste principal
112451	91.7 %	100 %	Umbral de confianza para motos
112535	65.2 %	65.2 %	Sin motos presentes (sin cambio)
112653	87.1 % ^a	96.8 %	Exclusión de zona de parqueo
112753	72.0 %	76.0 %	Umbral de confianza para motos
113819	63.6 % ^a	90.9 %	Umbral de confianza para motos
113930	36.4 %	45.5 %	Resultado mixto (ver discusión)

Nota. ^aLa precisión inicial de 112653 y 113819 en esta tabla corresponde a una recalibración previa a la incorporación del umbral diferenciado de motocicletas, por lo que difiere ligeramente de la precisión reportada en la Tabla 8.

En el video 113930 el conteo total mejoró, pero la clasificación mostró un resultado mixto: las motocicletas detectadas bajaron de tres a cero, mientras aumentaron las detecciones de buses y camiones. La inspección visual descartó una confusión sistemática de clase, atribuyendo el cambio a una reasignación de identificadores por parte de ByteTrack al reducirse el umbral global de confianza, sin evidencia de que las motocicletas fueran clasificadas erróneamente como vehículos de mayor tamaño.

Tras el reentrenamiento del modelo con el conjunto de datos ampliado (Sección de Desarrollo), el recall de motocicletas se incrementó del 24 % al 58 % (mAP_{50} de motocicleta 0.697, global 0.774), la mejora más significativa lograda durante el proceso de validación y ajuste del sistema.

Validación del Sistema de Estimación de Velocidad

La exactitud de la estimación de velocidad se evaluó registrando un vehículo de referencia a velocidad conocida (verificada por velocímetro y GPS) bajo las mismas condiciones de instalación de la cámara, en cuatro ubicaciones representativas. La Tabla 10 presenta los resultados.

Tabla 10*Validación de la estimación de velocidad con vehículo de referencia*

Video	Ubicación	Vel. real	Vel. medida	Error
Video 1	CC Cacique (original)	40 km/h	43.1 km/h	+7.8 %
Video 2	Bulevar	44 km/h	44.2 km/h	+0.5 %
Video 3	CC Cacique (paso elevado)	48 km/h	49.2 km/h	+2.5 %
Video 4	CC Cacique (paso elevado)	43 km/h	36.9 km/h	-14.2 %

Los errores oscilaron entre 0.5 % y 14.2 % en las tres primeras ubicaciones, consistente con la precisión esperada para un sistema basado en el cruce de dos líneas de referencia. El Video 4 presentó un error superior, atribuible a dos causas: vehículos estacionados en una zona de parqueo adyacente al carril generaron cruces ficticios con velocidades anómalamente bajas (esta corrida no contaba con el filtro de exclusión aplicado), y episodios de oclusión en tráfico congestionado generaron una probable reasignación de identificadores con velocidades anómalamente altas. Ambas causas son atribuibles al algoritmo de seguimiento y a una calibración incompleta de ese punto específico, no a errores del

modelo de detección.

Como validación adicional, se repitió el proceso ejecutando el modelo exportado en formato NCNN directamente sobre la Raspberry Pi 5, en lugar del modelo de referencia del equipo de desarrollo. Los resultados de los Videos 1 y 3 fueron prácticamente idénticos entre ambos formatos, respaldando la consistencia del proceso de exportación del modelo para su despliegue en el dispositivo embebido.

Desempeño Computacional en la Raspberry Pi 5

Para seleccionar el formato de modelo más adecuado para su ejecución en tiempo real sobre la Raspberry Pi 5, se compararon dos arquitecturas exportadas al formato NCNN, optimizado para inferencia en dispositivos con recursos limitados. La Tabla 11 resume los resultados obtenidos.

Tabla 11

Comparación de velocidad de inferencia sobre la Raspberry Pi 5 (formato NCNN)

Modelo	FPS
YOLO26n (NCNN)	55.9
YOLO11n (NCNN)	53.5

Aunque la diferencia en velocidad de inferencia entre ambos modelos fue moderada, YOLO26n presentó un mejor comportamiento de clasificación general, con una diferencia más marcada en la detección de motocicletas, por lo cual fue seleccionado como la versión desplegada del prototipo, en línea con la familia de arquitectura (YOLO26) elegida en el Capítulo de Desarrollo. Estos valores corresponden al modelo previo al proceso de reentrenamiento; dado que el reentrenamiento no modifica la arquitectura del modelo ni su proceso de exportación, no se esperan variaciones significativas en la velocidad de inferencia del modelo final utilizado en producción (`yolo26n_cacique_ncnn_model`).

Discusión General

Los resultados evidencian que SIMVIA alcanza precisión adecuada en la detección y el conteo de automóviles, con margen de mejora en motocicletas —donde se logró un incremento sustancial del recall mediante calibración y reentrenamiento—, y errores de velocidad reducidos en tráfico fluido, con un deterioro explicable en escenarios de mayor congestión u oclusión entre vehículos.

Un hallazgo transversal es que buena parte de los errores no corresponden al modelo de detección, sino al comportamiento del algoritmo de seguimiento ByteTrack ante oclusiones y a la necesidad de calibrar de forma independiente cada punto de instalación (líneas de referencia, distancia física, zona de exclusión de parqueo), más que a una limitación inherente al enfoque de detección empleado.

Estos resultados corresponden a las fases de validación con videos de referencia y grabaciones del sitio de despliegue; la validación en campo con el prototipo físico completo, instalado de forma permanente durante un periodo prolongado, constituye la tercera fase de esta estrategia.

Conclusiones

El desarrollo, la implementación y la validación experimental del sistema SIMVIA permiten formular las siguientes conclusiones, organizadas de acuerdo con los objetivos específicos planteados al inicio de este trabajo.

Respecto al hardware, la evaluación multicriterio confirmó que la Raspberry Pi 5 es una plataforma adecuada para ejecutar algoritmos de inteligencia artificial en tiempo real sin requerir hardware de aceleración adicional; combinada con la exportación de los modelos a NCNN, alcanzó velocidades de inferencia cercanas a 56 FPS, suficientes para el procesamiento continuo de video que exige el monitoreo vehicular en tiempo real.

En los algoritmos de visión computacional, YOLO26 —en su versión *Nano*, finalmente desplegada sobre la Raspberry Pi 5— mostró un desempeño adecuado para la detección, clasificación y conteo de vehículos, con un reconocimiento cercano al 100 % en la clase automóvil. La motocicleta fue el principal reto de clasificación, por su menor tamaño relativo y mayor variabilidad de poses; un umbral de confianza diferenciado y el reentrenamiento del modelo con datos adicionales permitieron incrementar su tasa de reconocimiento del 24 % al 58 %, una mejora sustancial aunque todavía con margen para ajustes futuros.

El sistema de estimación de velocidad alcanzó errores de entre 0.5 % y 14.2 % en condiciones de tráfico fluido, confirmando que la precisión depende ante todo de una calibración cuidadosa e independiente para cada punto de instalación, y no de una limitación inherente al modelo de detección empleado.

En el prototipo integral, el proceso de diseño industrial permitió construir una carcasa en acero galvanizado con un grado de protección equivalente al estándar IP55, validado mediante una prueba experimental de resistencia al ingreso de agua, y un sistema de ventilación controlado por PWM junto con un sensor de temperatura y humedad resolvió la gestión térmica del dispositivo dentro de la carcasa cerrada. El resultado es un prototipo portátil y de bajo costo (aproximadamente \$1.401.668 COP), removible y adaptable a distintos entornos urbanos sin requerir modificaciones permanentes a la infraestructura existente.

Finalmente, la estrategia de validación experimental permitió comprobar la viabilidad técnica del enfoque propuesto a lo largo de sus dos primeras fases —pruebas con videos de referencia en condiciones controladas y pruebas con grabaciones del sitio de despliegue—, que aportaron evidencia consistente sobre la precisión de detección, clasificación, conteo y estimación de velocidad del sistema. Queda pendiente una tercera fase —la instalación prolongada del prototipo físico completo en el sitio definitivo de despliegue—, planteada como continuación natural de este trabajo (véase Recomendaciones y Trabajo Futuro).

En conjunto, estos resultados muestran que es posible integrar, en un mismo sistema de monitoreo vehicular, procesamiento local de video, privacidad por diseño y bajo costo de implementación, sin depender de infraestructura en la nube ni comprometer el cumplimiento de la normativa colombiana de protección de datos personales. Este enfoque de procesamiento en el borde representa una alternativa viable frente a los sistemas convencionales de videovigilancia y aforo vehicular, particularmente para

ciudades intermedias como Bucaramanga, donde las restricciones presupuestales limitan la adopción de infraestructura de monitoreo más costosa. SIMVIA se consolida así como un aporte concreto al avance de las ciudades inteligentes y a la gestión sostenible de la movilidad urbana en contextos similares.

Recomendaciones y Trabajo Futuro

A partir de las limitaciones identificadas durante el desarrollo y la validación del sistema SIMVIA, se plantean las siguientes recomendaciones para trabajos futuros orientados a extender y mejorar la solución propuesta.

Se recomienda evaluar la incorporación de sensores de telemetría especializados, como LiDAR, como complemento al sistema de visión computacional actual: la estimación de velocidad basada exclusivamente en el cruce de dos líneas de referencia depende de una calibración precisa por cada sitio de instalación y es sensible a errores del algoritmo de seguimiento ante oclusiones entre vehículos, según se evidenció en el Capítulo de Resultados y Análisis. Un sensor de rango dedicado permitiría obtener mediciones de distancia y velocidad independientes del proceso de detección visual, mejorando potencialmente la exactitud del sistema en condiciones de tráfico congestionado.

Se recomienda igualmente ampliar el conjunto de datos utilizado para el entrenamiento del modelo, particularmente para la clase motocicleta: aunque los ajustes aplicados en este proyecto permitieron incrementar su tasa de reconocimiento del 24 % al 58 %, este valor continúa siendo inferior al alcanzado por las demás categorías vehiculares. Incorporar imágenes adicionales bajo diferentes condiciones de iluminación, clima y ángulos de captura podría contribuir a cerrar esta brecha de desempeño.

Finalmente, se recomienda explorar mecanismos de automatización del proceso de calibración, actualmente manual para cada ubicación (definición de las líneas de referencia, distancia física entre ellas y zona de exclusión de vehículos estacionados). El desarrollo de métodos de autocalibración, por ejemplo mediante objetos de referencia de dimensiones conocidas o técnicas de estimación automática de perspectiva a partir de la geometría de la escena, facilitaría la escalabilidad del sistema hacia múltiples puntos de monitoreo dentro de una misma ciudad.

Referencias

- Alcaldía de Bucaramanga. (2022). *Plan maestro de movilidad 2022–2037*. <https://www.bucaramanga.gov.co/>. ([Accessed 05-05-2026])
- Bamoumen, H., Temouden, A., Benamar, N., y Chtouki, Y. (2022). How TinyML can be leveraged to solve environmental problems: A survey. *IEEE 3ICT*.
- Congreso de Colombia. (2012). *Ley 1581 de 2012: Protección de datos personales*. <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981>. ([Accessed 05-05-2026])
- Gobierno de Singapur. (2020). *Smart mobility initiatives*. <https://www.smartnation.gov.sg/what-is-smart-nation/initiatives/Transport/smart-mobility/>. ([Accessed 05-05-2026])
- Goodfellow, I., Bengio, Y., y Courville, A. (2016). *Deep learning*. MIT Press.
- IBM. (2024). *¿Qué es la inteligencia artificial (IA)?* <https://www.ibm.com/mx-es/topics/artificial-intelligence>. ([Accessed 05-05-2026])
- Innovación Digital 360. (2023). *Sistemas embebidos: Qué son y para qué se utilizan*. <https://www.innovaciondigital360.com/iot/sistemas-embebidos-que-son-y-para-que-se-utilizan/>. ([Accessed 05-05-2026])
- Katariya, V., Jannat, F.-E., Pazho, A. D., Noghre, G. A., y Tabkhi, H. (2023). *VegaEdge: Edge AI confluence anomaly detection for real-time highway IoT-applications*. <https://arxiv.org/abs/2311.07880>. ([Accessed 14-07-2026])
- LeCun, Y., Bengio, Y., y Hinton, G. (2015). Deep learning. *Nature*, 521, 436–444.
- Liu, Y., Lu, Y., Shi, Q., y Ding, J. (2013). Optical flow based urban road vehicle tracking. *International Conference on Computational Intelligence and Security*.
- MinCIT. (2013). *Decreto 1377 de 2013*. <https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=53646>. ([Accessed 05-05-2026])
- NVIDIA. (2025). *Jetson developer kits*. <https://developer.nvidia.com/embedded-computing>. ([Accessed 05-05-2026])
- Redmon, J., Divvala, S., Girshick, R., y Farhadi, A. (2016). You only look once: Unified, real-time object detection. *CVPR*.
- Roboflow. (2020). *Train YOLOv4-tiny on custom data*. <https://blog.roboflow.com/train-yolov4-tiny-on-custom-data-lighting-fast-detection/>. ([Accessed 05-05-2026])
- Roboflow. (2026). *SIMVIA vehicle detection dataset*. <https://app.roboflow.com/simvia/car-detection-vciny-eg1o2/2>. ([Accessed 06-07-2026])
- RUNT. (2024). *Boletín de prensa 005 de 2024: El parque automotor de Colombia sigue creciendo*. <https://www.runt.com.co/>. ([Accessed 05-05-2026])
- Russell, S., y Norvig, P. (2020). *Artificial intelligence: A modern approach*. Pearson.
- Ruta N Medellín. (2020). *Proyectos de innovación en movilidad inteligente*. <https://www.rutanmedellin.org/>. ([Accessed 05-05-2026])
- Sarker, I. H. (2021). Deep learning: A comprehensive overview on techniques, taxonomy, applications and research directions. *SN Computer Science*, 2(420).

- Sarmiento Pilonieta, D. F., y Albornoz Villamil, S. A. (2026a). *Anexos del proyecto SIMVIA*. https://drive.google.com/drive/folders/17pgdPkkZd47S0xBEOZMQ_ufsW0VNBBb1?usp=sharing. ([Accessed 06-07-2026])
- Sarmiento Pilonieta, D. F., y Albornoz Villamil, S. A. (2026b). *SIMVIA: Sistema inteligente para monitoreo vehicular con inteligencia artificial*. <https://github.com/Danielsarmiento24/SIMVIA>. ([Accessed 06-07-2026])
- Secretaría de Movilidad de Bogotá. (2021). *Informe de gestión en movilidad inteligente*. https://www.movilidadbogota.gov.co/web/informes_de_gestion. ([Accessed 05-05-2026])
- Shaqib, S. M., Alo, A. P., Ramit, S. S., Rupak, A. U. H., Khan, S. S., y Rahman, M. S. (2024). *Vehicle speed detection system utilizing YOLOv8: Enhancing road safety and traffic management for metropolitan areas*. <https://arxiv.org/abs/2406.07710>. ([Accessed 14-07-2026])
- Szeliski, R. (2022). *Computer vision: Algorithms and applications*. Springer.
- Tan, T., y Cao, G. (2022). Deep learning on mobile devices through neural processing units and edge computing. *IEEE INFOCOM*.
- TfL. (2021). *Artificial intelligence and traffic management in London*. <https://tfl.gov.uk/corporate/publications-and-reports/technology-innovation>. ([Accessed 05-05-2026])
- Ultralytics. (2024). *YOLO11: Ultralytics YOLO11 docs*. <https://docs.ultralytics.com/models/yolo11/>. ([Accessed 05-05-2026])
- Ultralytics. (2025a). *Datasets overview*. <https://docs.ultralytics.com/datasets/detect/>. ([Accessed 05-05-2026])
- Ultralytics. (2025b). *Modelos YOLO de Ultralytics*. <https://docs.ultralytics.com/es/models/>. ([Accessed 05-05-2026])
- Ultralytics. (2025c). *Ultralytics YOLO documentation*. <https://docs.ultralytics.com/>. ([Accessed 05-05-2026])
- UNAL. (2023). *Diseño e implementación de un sistema de conteo vehicular basado en visión artificial*. Trabajo de grado. ([Accessed 05-05-2026])
- Univalle. (2024). *Sistema de monitoreo de tráfico vehicular mediante inteligencia artificial*. Trabajo de grado. ([Accessed 05-05-2026])
- Universidad Javeriana. (2023). *Aplicación de redes neuronales convolucionales para monitoreo vehicular urbano*. Trabajo de grado. ([Accessed 05-05-2026])
- Wang, C.-Y., Yeh, I.-H., y Liao, H.-Y. M. (2024). YOLOv9: Learning what you want to learn using programmable gradient information. *arXiv preprint arXiv:2402.13616*.